Xoddy IPDR Server

Installation Guide — Ubuntu 22.04 LTS

Xoddy IPDR Server is a MikroTik Traffic Flow (NetFlow / IPFIX) collector and IPDR system built to DOT IPDR rules. It ingests flow exports from MikroTik routers over UDP, stores them in PostgreSQL, lets operators search and export records, compresses old records on a schedule, and raises alerts when a NAS stops generating records.

This guide walks through a clean production install on a fresh Ubuntu 22.04 LTS server: the API server plus NetFlow collector runs as a systemd service, and the web dashboard is served by nginx, which also reverse-proxies API requests.

1. Prerequisites

- A server running Ubuntu 22.04 LTS with a user that has sudo privileges.
- Minimum 2 vCPU and 4 GB RAM (more for high flow volumes); ample disk for flow storage.
- Network reachability from your MikroTik routers to this server over UDP (default port 2055).
- Outbound internet access to install packages.

2. Update the system and install base packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git build-essential ufw ca-certificates gnupg
```

3. Install Node.js 24

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs
node -v   # should print v24.x
```

4. Enable pnpm

Node.js 24 ships with Corepack, which manages pnpm:

```
sudo corepack enable
corepack prepare pnpm@latest --activate
pnpm -v
```

5. Install PostgreSQL

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
```

6. Create the database and user

```
sudo -u postgres psql <<'SQL'
CREATE DATABASE xoddy_ipdr;
CREATE USER xoddy WITH ENCRYPTED PASSWORD 'change_this_password';
GRANT ALL PRIVILEGES ON DATABASE xoddy_ipdr TO xoddy;
\c xoddy_ipdr
GRANT ALL ON SCHEMA public TO xoddy;
SQL
```

> *Note: The GRANT ALL ON SCHEMA public line is required on PostgreSQL 15+ and harmless on the version 14 shipped with Ubuntu 22.04. Replace change_this_password with a strong password.*

7. Get the application code

```
sudo mkdir -p /opt/xoddy-ipdr
sudo chown $USER:$USER /opt/xoddy-ipdr
git clone https://github.com/pranaylokegaonkar/Xoddy-IPDR-Server.git /opt/xoddy-ipdr
cd /opt/xoddy-ipdr
```

8. Configure environment variables

Create an environment file the service and build steps will use:

```
cat > /opt/xoddy-ipdr/.env <<'ENV'
NODE_ENV=production
PORT=8080
DATABASE_URL=postgresql://xoddy:change_this_password@localhost:5432/xoddy_ipdr
SESSION_SECRET=REPLACE_WITH_RANDOM_VALUE
NETFLOW_PORT=2055
NETFLOW_HOST=0.0.0.0
NETFLOW_PUBLIC_HOST=YOUR_SERVER_PUBLIC_IP
ENV
```

> *Note: Generate a strong SESSION_SECRET with: openssl rand -hex 32. Set NETFLOW_PUBLIC_HOST to the address your MikroTik routers can reach — it is used in the config commands the app generates for each NAS.*

> *Note: If your database password contains special characters such as @ : / or #, they must be percent-encoded in DATABASE_URL. For example a password of Xoddy@123 must be written as Xoddy%40123 (@ becomes %40), otherwise the connection string is parsed incorrectly and the database connection fails.*

9. Install dependencies

```
cd /opt/xoddy-ipdr
pnpm install
```

10. Apply the database schema

```
set -a; source /opt/xoddy-ipdr/.env; set +a
pnpm --filter @workspace/db run push
```

11. Build the application

pnpm blocks dependency build scripts by default, which prevents esbuild (used to bundle the API server) from compiling. Approve the build scripts once before building:

```
cd /opt/xoddy-ipdr
pnpm approve-builds
```

> *Note: In the menu that appears, press a to select all packages (esbuild, @swc/core, msw, unrs-resolver), press Enter, then y to confirm. If pnpm approve-builds is unavailable, run: pnpm rebuild esbuild*

The web build also requires PORT and BASE_PATH to be set in the environment:

```
export BASE_PATH=/
export PORT=8080
pnpm run build
ls -la artifacts/api-server/dist/index.mjs
```

> *Note: This produces the API server bundle at artifacts/api-server/dist/index.mjs and the static web UI at artifacts/xoddy-ipdr/dist/public. Confirm index.mjs exists before continuing — if it is missing, the build did not finish (re-run the approve-builds step above).*

12. Run the API server and collector as a service

```
sudo tee /etc/systemd/system/xoddy-ipdr.service > /dev/null <<'UNIT'
[Unit]
Description=Xoddy IPDR Server (API + NetFlow collector)
After=network.target postgresql.service

[Service]
Type=simple
WorkingDirectory=/opt/xoddy-ipdr
EnvironmentFile=/opt/xoddy-ipdr/.env
ExecStart=/usr/bin/node --enable-source-maps artifacts/api-server/dist/index.mjs
Restart=always
RestartSec=5
User=www-data
Group=www-data

[Install]
WantedBy=multi-user.target
UNIT
```

```
sudo chown -R www-data:www-data /opt/xoddy-ipdr
sudo systemctl daemon-reload
sudo systemctl enable --now xoddy-ipdr
sudo systemctl status xoddy-ipdr --no-pager
```

13. Serve the web UI with nginx

```
sudo apt install -y nginx
sudo tee /etc/nginx/sites-available/xoddy-ipdr > /dev/null <<'NGINX'
server {
    listen 80;
    server_name _;
    root /opt/xoddy-ipdr/artifacts/xoddy-ipdr/dist/public;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
NGINX
```

```
sudo ln -sf /etc/nginx/sites-available/xoddy-ipdr /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

14. Configure the firewall

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 2055/udp
sudo ufw enable
sudo ufw status
```

15. Verify the installation

```
curl http://localhost/api/healthz
```

Expect a JSON health response. Then open http://YOUR_SERVER_IP/ in a browser to view the dashboard.

16. Point a MikroTik router at the collector

In the dashboard, open NAS Devices, add your router, and copy the generated config. Alternatively run these on the router (RouterOS terminal):

```
/ip traffic-flow set enabled=yes interfaces=all active-flow-timeout=1m inactive-flow-
timeout=15s
/ip traffic-flow target add dst-address=YOUR_SERVER_PUBLIC_IP port=2055 version=9 v9-
template-refresh=20 v9-template-timeout=30m
```

> *Note: version can be 1, 5, 9, or ipfix. For v9 / IPFIX the router must send its template first, so records may take a couple of minutes to appear after enabling.*

17. Updating to a new version later

```
cd /opt/xoddy-ipdr
git pull
pnpm install
export BASE_PATH=/ PORT=8080
pnpm run build
set -a; source /opt/xoddy-ipdr/.env; set +a
pnpm --filter @workspace/db run push
sudo systemctl restart xoddy-ipdr
sudo systemctl reload nginx
```

18. Common deployment errors and fixes

nginx returns 502 Bad Gateway on /api and the service keeps restarting; journalctl shows: Cannot find module .../dist/index.mjs

Cause: the build did not finish because pnpm blocked esbuild's build script (ERR_PNPM_IGNORED_BUILDS: Ignored build scripts: esbuild). Fix it by approving the scripts and rebuilding:

```
cd /opt/xoddy-ipdr
pnpm approve-builds        # press a, Enter, then y  (or: pnpm rebuild esbuild)
export BASE_PATH=/ PORT=8080
pnpm run build
ls -la artifacts/api-server/dist/index.mjs
sudo chown -R www-data:www-data /opt/xoddy-ipdr
sudo systemctl restart xoddy-ipdr
sudo systemctl status xoddy-ipdr --no-pager
```

Database connection fails even though the password is correct.

Cause: a special character in the password is not percent-encoded in DATABASE_URL. Encode @ as %40 (also : as %3A, / as %2F, # as %23), then re-run the schema push and restart:

```
# e.g. password Xoddy@123 must be written Xoddy%40123
sudo nano /opt/xoddy-ipdr/.env
set -a; source /opt/xoddy-ipdr/.env; set +a
pnpm --filter @workspace/db run push
sudo systemctl restart xoddy-ipdr
```

nginx returns 500 Internal Server Error on the root page (/).

Cause: nginx (running as www-data) cannot read the built UI files. Fix ownership and directory traversal permissions:

```
sudo chown -R www-data:www-data /opt/xoddy-ipdr
sudo chmod o+x /opt
sudo systemctl reload nginx
```

19. Troubleshooting

- View service logs in real time: sudo journalctl -u xoddy-ipdr -f
- Database connection errors: confirm DATABASE_URL is correct and PostgreSQL is running (sudo systemctl status postgresql).
- No flows appearing: ensure UDP 2055 is open, the NAS is enabled in the app, and the router targets the correct server IP.
- Web build fails: make sure BASE_PATH and PORT are exported before running pnpm run build.
- nginx returns 502: the API service is not running — check sudo systemctl status xoddy-ipdr.